

Verificação, Validação e Teste de Software (VVT) 2026.1

Vamos operacionalizar aspectos de qualidade e não somente executar testes.

Week 1 - Dia 0 - *Software Engineering RECAP*
Prof. Dr. Awdren Fontão



UNIVERSIDADE
FEDERAL DE
MATO GROSSO DO SUL



Professor

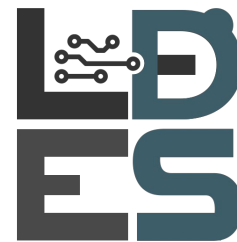
Dr. Awdren de Lima Fontão

awdren.fontao@ufms.br

Áreas de Pesquisa: SE with/for AI,
DevRel/DevX, Software Ecosystem

<https://awdren.github.io/>

<http://lattes.cnpq.br/0597440372595970>



Antes de tudo: alinhamento

- Postura em sala de aula;
 - Postura com colegas;
 - Compromisso em nível profissional;
 - Não deixe para cima da hora;
 - Negociar é habilidade e pode ser exercitada na disciplina;
 - Quero ajudar na construção de jornadas de pessoas de engenharia
-
- **Conformidade, clarificação, cultura e conexão.**



Próximos da indústria e da academia

Trabalharemos por semanas - Week 0, Week 1... Week N

Jornada em pares - *Buddies*

Avaliações (?) com casos de cenários reais

Hackathon de Teste de Software

Conteúdos como podcasts, vídeos e artigos para cada dia

~~Não~~ iremos usar GenAI



A disciplina

1. Plano de Ensino já no SISCAD e AVA;
2. Moodle: ava.ufms.br – verificar se já está com acesso;
3. Cronograma da disciplina;
4. Discord (voltarei após não ter utilizado no semestre anterior :S);
5. Semanal: 4 horas;
6. Semestral: 68 horas (34 horas práticas e 34 horas presenciais), sendo 60 horas ofertadas presencialmente e 8 horas ofertadas a distância.
7. Atendimento.

Objetivos de hoje

Apresentar o **plano de Ensino**;

Revisar brevemente conceitos importantes da **Engenharia de Software**;

Discutir o conceitos de **verificação**, **validação** e **teste** de software;

Motivar para o estudo dos temas envolvidos na disciplina.





**Não é aceitável imaginar
a construção de um
sistema de software
sem a construção de
uma suite de testes
automatizados...**

Expectativas



Conhecimento Teórico e Prático

Adquirir **conhecimento teórico e prático**, destacando a necessidade de compreensão conceitual e **aplicação prática**. Há interesse em aprender **estratégias, técnicas e ferramentas** de testes para garantir a **qualidade** do software.



Cenários Reais de Mercado

Entender **como os testes são usados na prática**, considerando **cenários reais de mercado e trade-offs** das técnicas. Essa expectativa reforça a necessidade de uma **abordagem prática e contextualizada**.



Carreira e Contribuição

Uso no **ambiente de trabalho, preparação para concursos e contribuição em projetos** acadêmicos e profissionais.

SOFTWARE



modelo
mental



modelo
tecnológico



código como artefato de vínculo

Como guiar uma intenção em código?

Engenharia

é a aplicação de princípios matemáticos, científicos e empíricos para projetar, construir, operar e manter estruturas, máquinas, processos e sistemas de forma eficiente e econômica.



Engenharia de Software

“... é a aplicação de uma abordagem sistemática,
disciplinada e quantificável, para o
desenvolvimento, operação e manutenção do
software...” (IEEE)

Engenharia de Software não é só
desenvolvimento!



Shanghai Tower é um arranha-céu construído em Xangai, na China.

O edifício possui 128 andares – e mais cinco pavimentos no porão – distribuídos em 632 metros de altura.

Um dos mais altos decks de observação panorâmica do mundo está situado no edifício

Software é uma das mais
complexas construções humanas.

Bancos, comércio, transporte, hotéis, mídia, educação,
saúde, lazer, moeda, etc

X = empresa & app

Toda empresa está se transformando em uma
empresa de software -> Data Driven -> AI Driven...

X = Uber, iFood, Zoom, YouTube, Inter, Nubank, etc.

Empresa de software $\Rightarrow$ movida por **engenheiros de software**



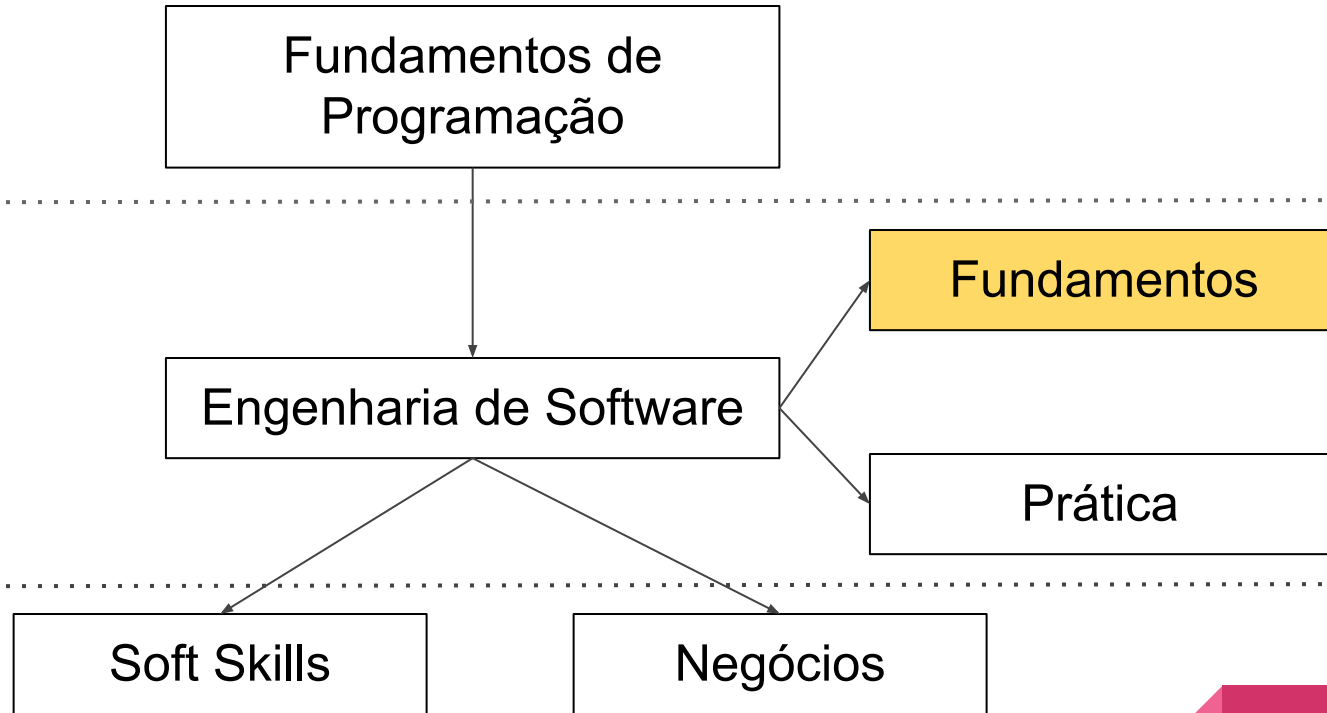
GitLab team photo, New Orleans, 2019

O que eu preciso para ser um excelente Engenheiro de Software?

“Queremos alunos e professores que pensem como
Engenheiros de Software!”

Inteligência Artificial, Segurança Cibernética, Computação em
Nuvem e Internet das Coisas...

Profissionais éticos e responsáveis



“Muitas vezes, contratamos um dev pelos seus hard skills, mas somos forçados a demiti-lo por causa dos soft skills”

“Eu não quero desenvolvedor só tarefeiro...”

Programming in the small vs Programming in the large



Programação



Engenharia de software

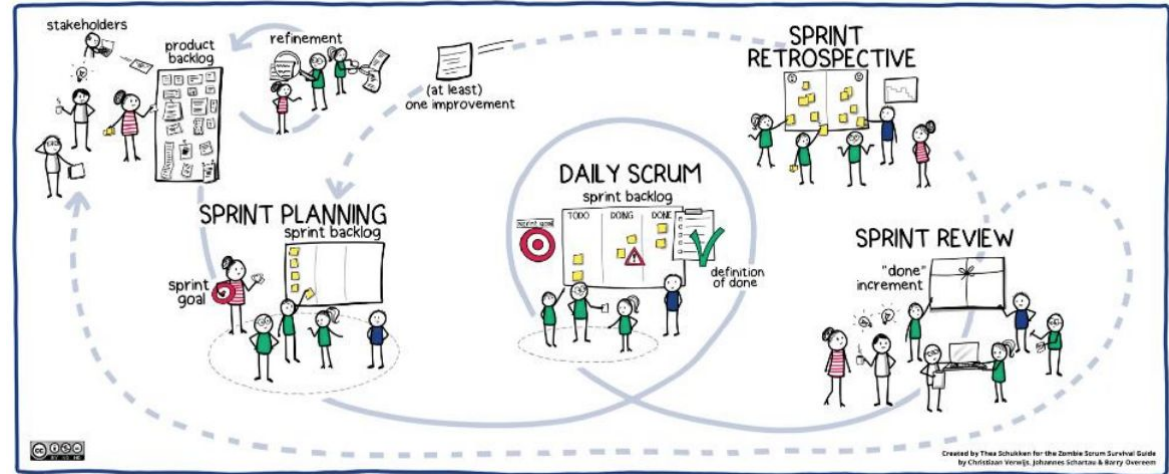
Programação (programming in the small)

- Um problema, algoritmo ou funcionalidade
- Um desenvolvedor
- Poucas linhas de código (~KLOC)
- Pouca ou nenhuma manutenção futura

Engenharia de Software (programming in the large)

- Dezenas de funcionalidades
- Milhões de linhas de código
- Dezenas de desenvolvedores
- Diversos clientes, versões e tecnologias
- Interfaces e integrações externas
- Manutenção e operação por anos
- Requisitos não-funcionais (segurança, escalabilidade, etc)
- Etc...

- Dezenas de desenvolvedores ⇒ **Processos/Abordagens**
 - Extreme Programming (XP)
 - Scrum
 - Kanban



- Dezenas de funcionalidades $\Rightarrow$ **Requisitos**

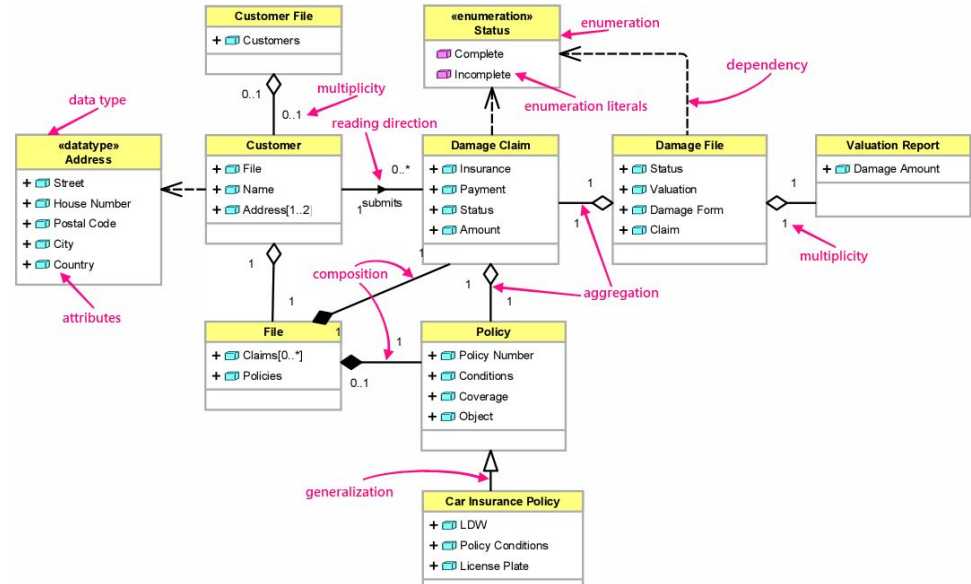
- Histórias de Usuários
- Casos de Uso
- MVPs
- Testes A/B



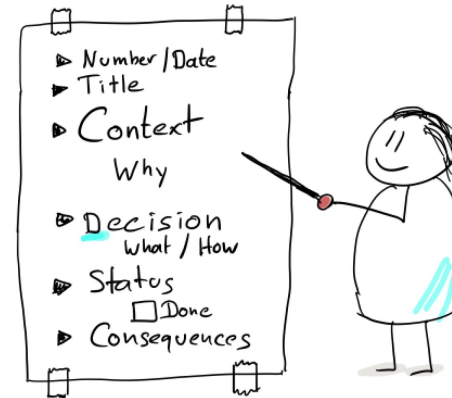
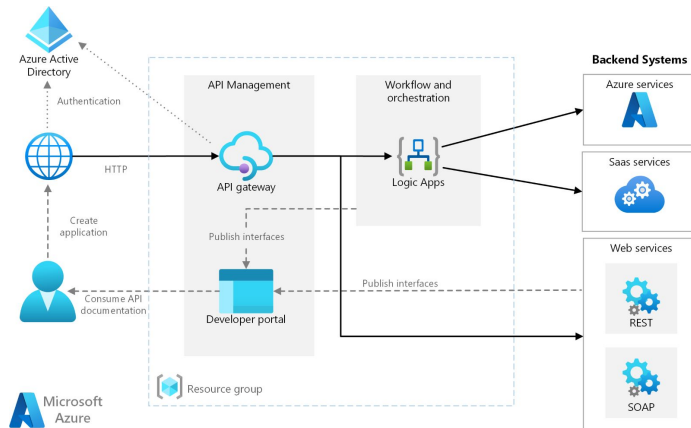
- Milhões de LOC $\Rightarrow$ **Modelos**

- Modelos gráficos (“uma figura vale mais do que mil palavras”)

- Diagramas UML



- Milhões de LOC ⇒ **Projeto & Arquitetura** (caps. 5, 6 e 7)
 - Princípios de Projeto: coesão & acoplamento; SOLID
 - Padrões de Projeto: singleton, Proxy, Adaptadores, etc
 - Arquitetura: camadas, MVC, microsserviços, Pub/Sub, ADRs, etc



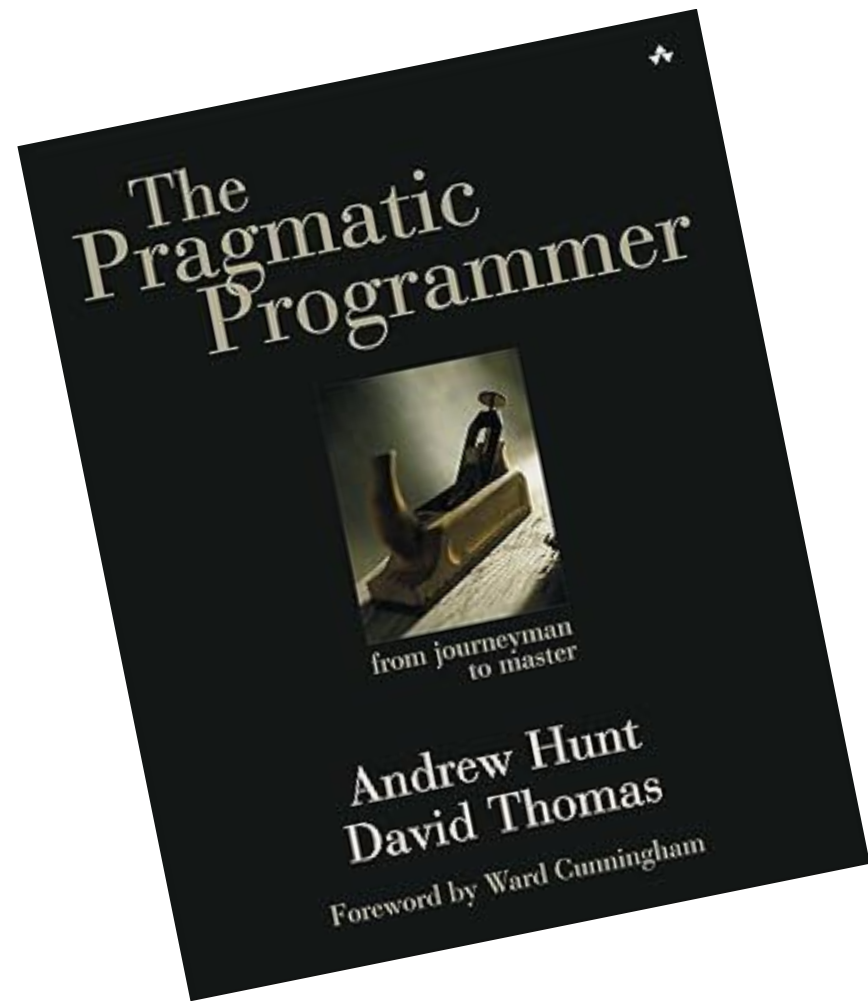
- Mudanças constantes ⇒ **Testes Automatizados**
 - Testes de unidade
 - Testes de integração
 - Testes de sistema
 - Teste de imersão
 - Teste funcional

```
public class Math {  
    public int fatorial(int n) {  
        int fat = 1;  
        for (int i = 1; i <= n; i++)  
            fat = fat * i;  
        return fat;  
    }  
}
```

Qual código falta
aqui?

```
public class Math {  
    public int fatorial(int n) {  
        int fat = 1;  
        for (int i = 1; i <= n; i++)  
            fat = fat * i;  
        return fat;  
    }  
}
```

```
public class MathTest {  
    @Test  
    public void testFatorial() {  
        Math m = new Math();  
        assertEquals(1, m.fatorial(0));  
        assertEquals(1, m.fatorial(1));  
        assertEquals(2, m.fatorial(2));  
        assertEquals(6, m.fatorial(3));  
    }  
}
```



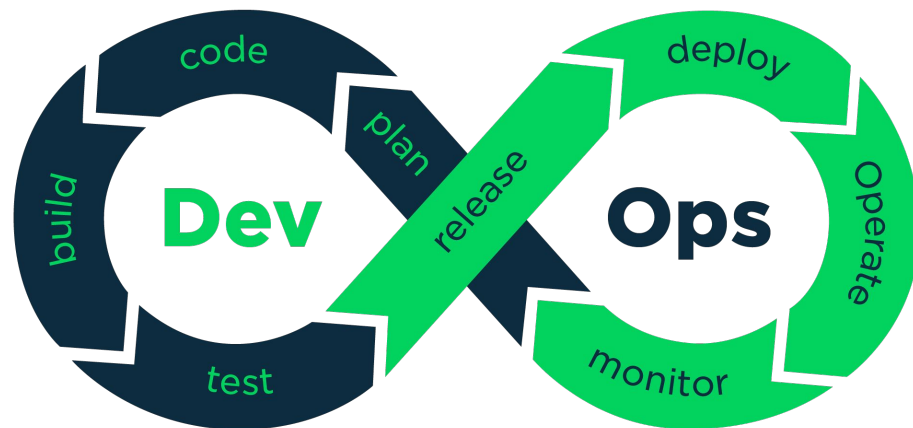
*“Test your software, or your users
will”*

(The Pragmatic Programmer, Tip 70)

- Usado por anos ⇒ **Manutenção e evolução**
 - Catálogos de refactoring
 - Code smells



- Temos também que operar $\Rightarrow$ **DevOps, MLOps, AIOps... *Ops**
 - Controle de versões
 - Gerenciamento de branches
 - Integração Contínua
 - Deployment contínuo
 - etc



Mais uma comparação para tentar explicar
o trabalho de um ES

Explorando um pouco mais...

- CG funciona 24 horas → idem software (operação)
- CG tem diversos bairros → módulos, camadas, etc
- CG tem que se integrar com municípios vizinhos → integração
- CG precisa de um plano de expansão → novas features
- CG tem problemas de segurança → idem para software
- CG tem milhares de funcionários → idem empresas software
- CG tem problemas de limpeza → clean code, qualidade

4 T's da Engenharia de Software

Time

alinhar pessoas, processos,
trabalho em equipe

Tempo

sistemas duram anos;
manutenção, evolução

Tamanho

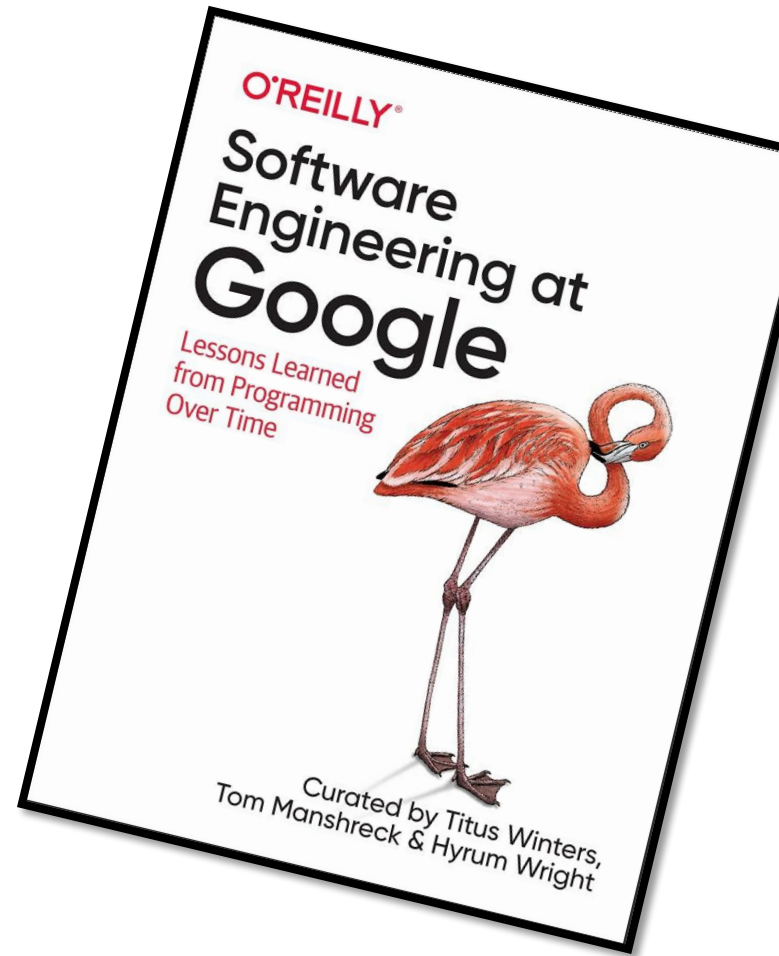
sistemas têm MLOC;
complexidade

Tecnologia

ferramenta de trabalho;
evolução constante; legado

Mas o que é SE?

- É a prática de desenvolver, modificar e manter software ao longo do tempo, levando em conta escalabilidade, sustentabilidade e trade-offs. *Diferente da programação isolada, envolve colaboração em equipe, decisões estratégicas e adaptação contínua a mudanças tecnológicas e organizacionais.* Como dito no Google: "Software engineering is programming integrated over time."



SE4AI vs. AI4SE

SE4AI



A aplicação de métodos de engenharia para construir, operar e manter sistemas de IA de forma robusta e segura.

AI4SE



O uso de técnicas de IA para apoiar tarefas de engenharia de software (como automação de testes).

Aquecimento:

Planeje como garantia a qualidade deste software



O “TrilhaSegura” é um aplicativo móvel projetado para entusiastas de trilhas e caminhadas. Seu objetivo é fornecer informações essenciais, dicas de segurança e recursos úteis para tornar a experiência de trilha mais agradável e segura.

10 minutos.

Procure um buddy.